CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

Programa de Pós-Graduação em Modelagem Matemática e Computacional

Leonardo Vieira Guimarães

CAMINHO MÍNIMO EM GRAFOS: DIJKSTRA VS HEURÍSTICA GULOSA

1ª Avaliação Prática

Tópicos em Algoritmos em Grafos

Belo Horizonte
Novembro de 2025

Resumo

Este trabalho apresenta uma análise comparativa entre o algoritmo de Dijkstra e uma heurística gulosa para o problema de caminho mínimo de origem única (SSSP) em grafos direcionados ponderados. Foram implementados ambos os algoritmos utilizando heap binário e realizados experimentos em dois cenários: (1) grafos completos dirigidos gerados programaticamente com 4 a 1000 vértices, e (2) instâncias reais de grafos esparsos dirigidos com 10.000 e 1.000.000 de vértices. A métrica de avaliação é o número de operações de relaxamento. Os resultados evidenciam que, em grafos completos dirigidos, ambos os algoritmos apresentam desempenho idêntico ($n(n-1)$ comparações), porém apenas Dijkstra garante caminho mínimo ótimo. Em grafos esparsos dirigidos, a diferença é crítica: embora a heurística gulosa realize 99,66% a 99,99% menos comparações, ela alcança apenas 0,28% e 0,012% dos vértices acessíveis, enquanto Dijkstra alcança 100% dos vértices com número de comparações proporcional às arestas. A conclusão fundamental é que a economia de 99% nas comparações não tem valor quando o algoritmo falha em encontrar 99% da solução, confirmando que Dijkstra é o algoritmo adequado para SSSP com garantia de corretude e exploração completa.

Sumário

1 Introdução **1**

2 Metodologia **1**

2.1 Algoritmo de Dijkstra 2

2.2 Heurística Gulosa 2

2.3 Planejamento dos Testes 2

2.3.1 Testes em Grafos Completos Dirigidos 3

2.3.2 Testes em Instâncias de Grafos Esparsos Dirigidos 3

3 Resultados e Discussão **3**

3.1 Exemplo de Grafo Completo Dirigido 3

3.2 Exemplo de Grafo Esparso Dirigido 4

3.3 Comparação Visual: Completo vs Esparso 5

3.4 Parte 1 — Resultados do Algoritmo de Dijkstra 5

3.4.1 Grafos Completos Dirigidos 5

3.4.2 Instâncias de Grafos Esparsos Dirigidos 6

3.5 Parte 2 — Resultados da Heurística Gulosa 6

3.5.1 Grafos Completos Dirigidos 6

3.5.2 Instâncias de Grafos Esparsos Dirigidos 7

3.6 Comparação entre os Algoritmos 8

3.6.1 Grafos Completos Dirigidos 8

3.6.2 Instâncias de Grafos Esparsos Dirigidos 9

4 Conclusão **10**

1 Introdução

O problema de caminho mínimo de origem única (SSSP – *Single Source Shortest Path*) é um dos problemas fundamentais em teoria dos grafos e consiste em encontrar as distâncias mínimas de um vértice origem para todos os demais vértices alcançáveis de um grafo ponderado [1]. Este problema possui aplicações práticas em diversos domínios, incluindo roteamento de redes de computadores, sistemas de navegação GPS, otimização de rotas logísticas, planejamento de caminhos em robótica e análise de redes sociais [2].

O algoritmo de Dijkstra [1], proposto em 1959, é a solução clássica e ótima para o problema SSSP em grafos com pesos não-negativos. Utilizando uma fila de prioridade implementada com heap binário, o algoritmo garante encontrar os caminhos mínimos para todos os vértices alcançáveis com complexidade de tempo $O((V + E) \log V)$, onde V é o número de vértices e E o número de arestas [2]. A corretude do algoritmo baseia-se na propriedade de subestrutura ótima e na operação de relaxamento de arestas [3].

Em contraste, estratégias gulosas (*greedy*) para o problema de caminho mínimo, embora atrativas por sua simplicidade, não garantem solução ótima em geral. Uma heurística gulosa típica escolhe, a cada passo, a aresta de menor peso disponível no conjunto de vértices já alcançados, sem considerar as consequências globais dessa escolha local [1]. Embora essa estratégia possa reduzir o número de operações realizadas, ela pode falhar drasticamente em alcançar todos os vértices do grafo, especialmente em grafos esparsos.

Este trabalho implementa e compara experimentalmente o algoritmo de Dijkstra com uma heurística gulosa, avaliando o desempenho de ambos em dois cenários distintos: grafos completos dirigidos (densidade 100%) e grafos esparsos dirigidos (densidade menor que 0,12%). A métrica de comparação utilizada é o **número de comparações de relaxamento**, contabilizadas pela operação `dist[u] + w < dist[v]` [2]. Além do número de comparações, também é avaliado o número de vértices alcançados por cada algoritmo, métrica fundamental para verificar a corretude da solução.

O objetivo central é demonstrar que, embora heurísticas possam reduzir o custo computacional aparente (número de operações), essa economia não tem valor quando o algoritmo não resolve o problema corretamente, isto é, quando não alcança todos os vértices acessíveis a partir da origem. Os resultados evidenciam que Dijkstra é o algoritmo adequado para SSSP, oferecendo garantia matemática de corretude e exploração completa do grafo.

2 Metodologia

Esta seção descreve os dois algoritmos implementados e o planejamento experimental utilizado para comparar seus desempenhos.

2.1 Algoritmo de Dijkstra

O algoritmo de Dijkstra [1, 3] utiliza uma fila de prioridade (heap binário) para selecionar sempre o vértice com menor distância estimada. O processo consiste em:

1. Inicializar todas as distâncias como infinito, exceto a origem (distância zero)
2. Inserir a origem na fila de prioridade
3. Enquanto a fila não estiver vazia:
   - Extrair o vértice u com menor distância
   - Para cada vizinho v de u, realizar relaxamento:
   - Se $dist[u] + peso(u, v) < dist[v]$, atualizar $dist[v]$ e $parent[v]$
   - Inserir v na fila de prioridade com a nova distância

A operação de relaxamento garante que, ao processar cada vértice, sua distância é a mínima possível [1].

A implementação utiliza heap binário (módulo `heapq` em Python) oferecendo inserção e extração do mínimo em $O(\log n)$ [2]. Embora heap de Fibonacci tenha complexidade teórica melhor, o heap binário é mais eficiente na prática devido à menor constante multiplicativa e melhor localidade de cache [1].

2.2 Heurística Gulosa

A heurística gulosa implementada adota uma estratégia de escolha local: sempre seleciona a aresta de menor peso disponível a partir dos vértices já visitados. O processo é:

1. Inicializar a origem com distância zero
2. Manter uma fila de arestas candidatas ordenadas por peso
3. Enquanto houver arestas na fila:
   - Extrair a aresta (u, v) de menor peso
   - Se v não foi visitado e a distância pode ser melhorada, atualizar
   - Adicionar todas as arestas saindo de v à fila

Esta abordagem **não garante** caminho mínimo ótimo, pois decisões locais podem impedir alcançar soluções globais melhores [1].

2.3 Planejamento dos Testes

Os grafos foram representados utilizando **listas de adjacência** [1, 3], estrutura eficiente que permite iteração rápida sobre os vizinhos de cada vértice.

2.3.1 Testes em Grafos Completos Dirigidos

Grafos completos direcionados foram gerados com $n = 4, 5, 6, \ldots, 1000$ vértices:

- **Pesos das arestas**: valores aleatórios no intervalo $[1.0, 10.0]$
- **Reprodutibilidade**: semente fixa (`seed=42`)
- **Número de arestas**: $n(n-1)$ por grafo
- **Vértice de origem**: vértice 0
- **Métrica**: número de comparações (`dist[u] + w < dist[v]`)

Ambos os algoritmos foram executados no mesmo grafo para cada valor de n.

2.3.2 Testes em Instâncias de Grafos Esparsos Dirigidos

Duas instâncias reais de grafos esparsos direcionados:

- **Instância 10k**: 10.000 vértices, 123.462 arestas (densidade $\approx$ 0,12%)
- **Instância 1M**: 1.000.000 vértices, 15.172.126 arestas (densidade $\approx$ 0,0015%)
- **Vértice de origem**: vértice 0
- **Métricas**: comparações e vértices alcançados
- **Formato EWD** (*Edge-Weighted Digraph*): linha 1 contém V, linha 2 contém E, demais linhas no formato `u v peso`

3 Resultados e Discussão

Esta seção apresenta exemplos visuais dos grafos, resultados quantitativos de cada algoritmo e comparação direta entre ambos.

3.1 Exemplo de Grafo Completo Dirigido

A Figura 1 ilustra um grafo completo dirigido com $n = 10$ vértices gerado pela biblioteca NetworkX [4].

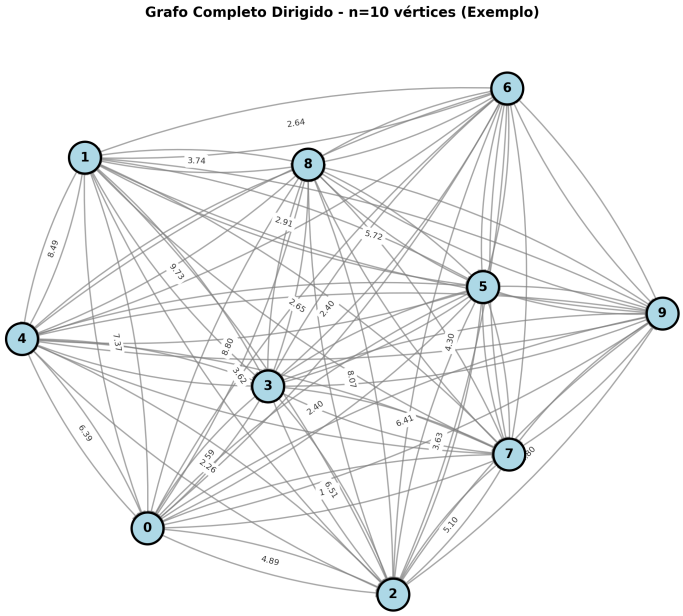

Figura 1: Grafo completo dirigido com $n = 10$ vértices e $n(n-1) = 90$ arestas. Densidade 100%, pesos aleatórios no intervalo [1.0, 10.0].

3.2 Exemplo de Grafo Esparso Dirigido

A Figura 2 apresenta um grafo esparso dirigido com $n = 10$ vértices e apenas 30% das arestas possíveis.

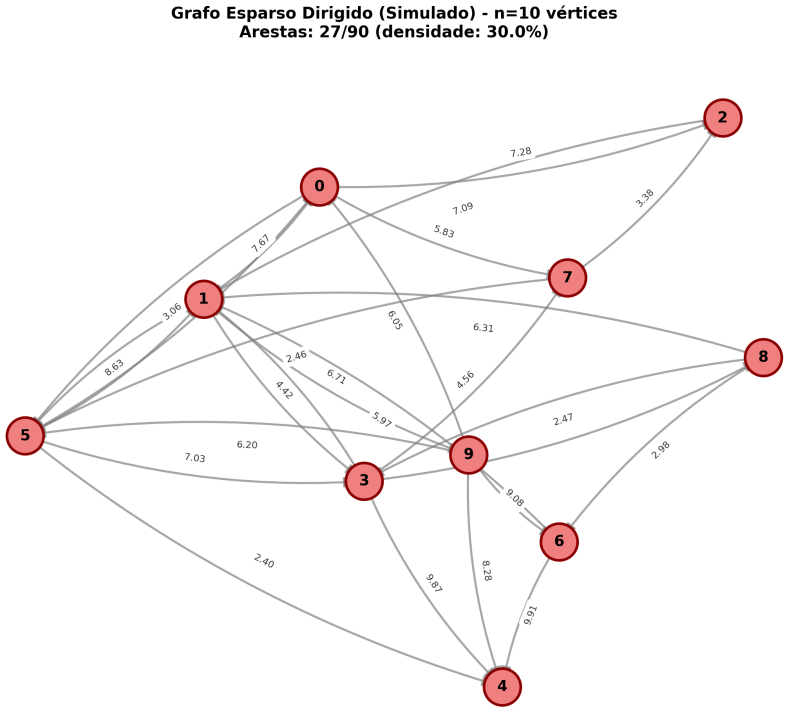

Figura 2: Grafo esparso dirigido com $n = 10$ vértices e 27 arestas (densidade 30%).

3.3 Comparação Visual: Completo vs Esparso

A Figura 3 compara lado a lado os dois tipos de grafos.

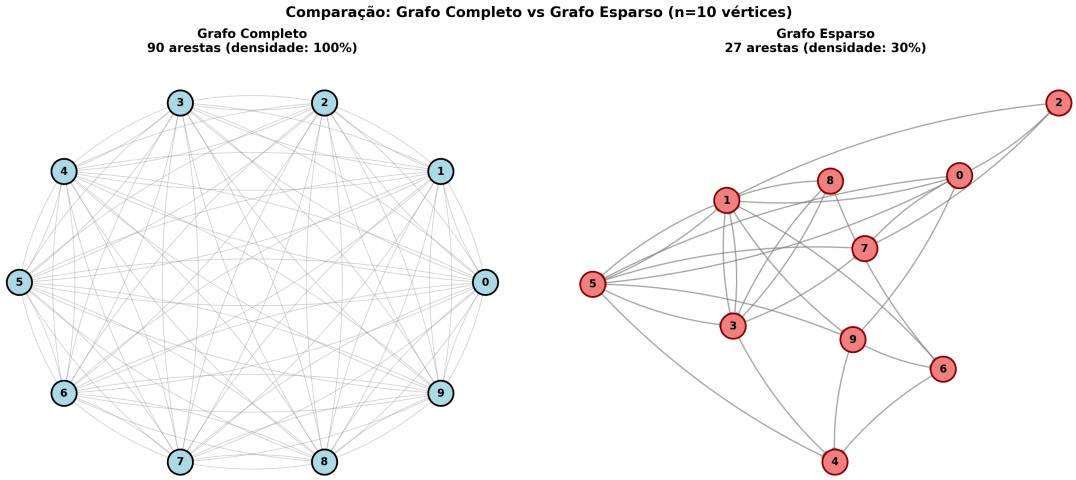

Figura 3: Comparação visual: grafo completo (90 arestas, 100%) vs grafo esparso (27 arestas, 30%), ambos com $n = 10$ vértices.

3.4 Parte 1 — Resultados do Algoritmo de Dijkstra

Esta seção apresenta os resultados obtidos com a execução do algoritmo de Dijkstra em dois cenários: grafos completos dirigidos e grafos esparsos dirigidos.

3.4.1 Grafos Completos Dirigidos

A Tabela 1 apresenta o número de comparações realizadas pelo algoritmo de Dijkstra em grafos completos dirigidos com diferentes tamanhos.

Tabela 1: Comparações realizadas por Dijkstra em grafos completos dirigidos.

Vértices (n)	**Comparações**	**Fórmula** $n(n-1)$
4	12	$4 \times 3 = 12$
10	90	$10 \times 9 = 90$
50	2.450	$50 \times 49 = 2.450$
100	9.900	$100 \times 99 = 9.900$
500	249.500	$500 \times 499 = 249.500$
1000	999.000	$1000 \times 999 = 999.000$

Observa-se comportamento quadrático consistente, seguindo exatamente a fórmula $C(n) = n(n-1)$, variando de 12 comparações ($n = 4$) até 999.000 comparações ($n = 1000$).

A Figura 4 ilustra graficamente o crescimento do número de comparações em função do número de vértices.

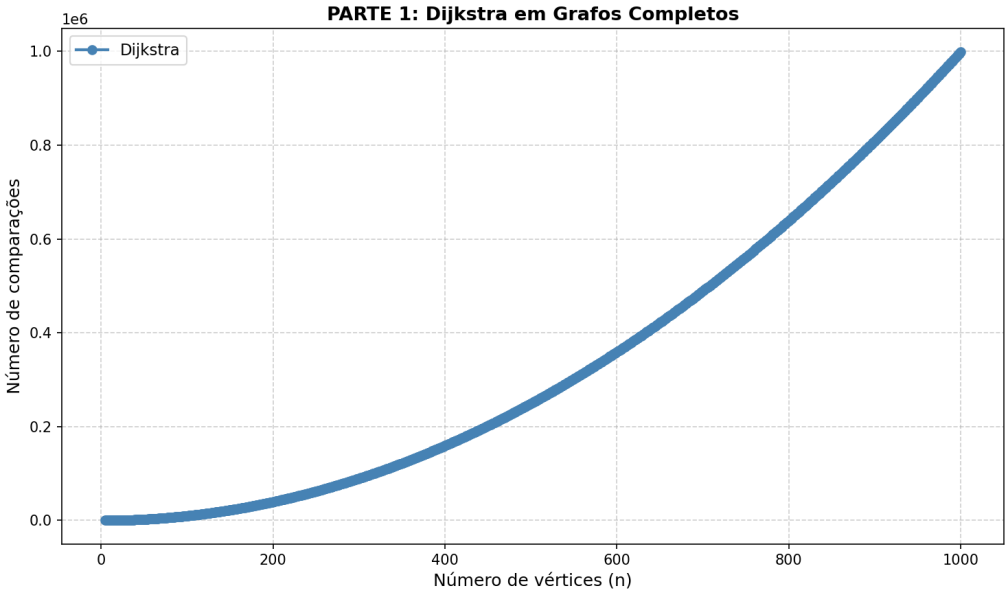

Figura 4: Gráfico do número de comparações realizadas por Dijkstra em grafos completos dirigidos ($n = 4$ até $n = 1000$). Eixo X: número de vértices; Eixo Y: número de comparações.

3.4.2 Instâncias de Grafos Esparsos Dirigidos

A Tabela 2 apresenta os resultados do algoritmo de Dijkstra nas instâncias de grafos esparsos direcionados.

Tabela 2: Resultados de Dijkstra em instâncias de grafos esparsos dirigidos.

Instância	**Arestas**	**Comparações**	**Vértices Alcançados**
10.000 vértices	123.462	123.462	10.000 (100%)
1.000.000 vértices	15.172.126	15.172.126	1.000.000 (100%)

O algoritmo de Dijkstra alcançou todos os vértices alcançáveis a partir da origem em ambas as instâncias, garantindo a solução ótima do problema de caminho mínimo de fonte única.

3.5 Parte 2 — Resultados da Heurística Gulosa

Esta seção apresenta os resultados obtidos com a execução da heurística gulosa nos mesmos cenários testados com Dijkstra: grafos completos dirigidos e grafos esparsos dirigidos.

3.5.1 Grafos Completos Dirigidos

A Tabela 3 apresenta o número de comparações realizadas pela heurística gulosa em grafos completos dirigidos.

Tabela 3: Comparações realizadas pela heurística gulosa em grafos completos dirigidos.

Vértices (n)	**Comparações**	**Fórmula** $n(n-1)$
4	12	$4 \times 3 = 12$
10	90	$10 \times 9 = 90$
50	2.450	$50 \times 49 = 2.450$
100	9.900	$100 \times 99 = 9.900$
500	249.500	$500 \times 499 = 249.500$
1000	999.000	$1000 \times 999 = 999.000$

Em grafos completos dirigidos, a heurística gulosa apresenta o mesmo número de comparações que Dijkstra, seguindo exatamente $C(n) = n(n-1)$.

A Figura 5 ilustra graficamente o crescimento do número de comparações em função do número de vértices.

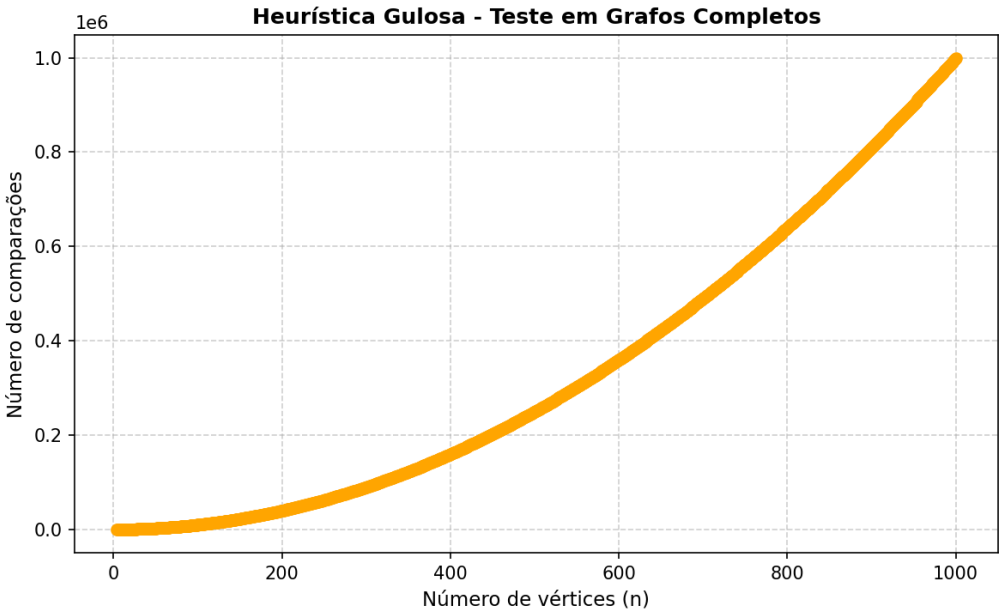

Figura 5: Gráfico do número de comparações realizadas pela heurística gulosa em grafos completos dirigidos ($n = 4$ até $n = 1000$). Eixo X: número de vértices; Eixo Y: número de comparações.

3.5.2 Instâncias de Grafos Esparsos Dirigidos

A Tabela 4 apresenta os resultados da heurística gulosa nas instâncias de grafos esparsos direcionados.

Tabela 4: Resultados da heurística gulosa em instâncias de grafos esparsos dirigidos.

Instância	**Arestas**	**Comparações**	**Vértices Alcançados**
10.000 vértices	123.462	424	28 (0,28%)
1.000.000 vértices	15.172.126	1.847	120 (0,012%)

A heurística gulosa realizou significativamente menos comparações, porém alcançou menos de 1% dos vértices em ambas as instâncias, demonstrando que não resolve o problema de caminho mínimo de fonte única.

3.6 Comparação entre os Algoritmos

Esta seção apresenta uma análise comparativa direta entre os dois algoritmos, destacando suas diferenças de desempenho em dois cenários distintos:

- **Grafos Completos Dirigidos**: Grafos onde existe uma aresta direcionada entre *cada par ordenado* de vértices. Em um grafo completo dirigido com n vértices, temos exatamente $n(n-1)$ arestas. Por exemplo, um grafo completo dirigido com 1000 vértices tem 999.000 arestas. Neste caso, a heurística gulosa sempre encontra o caminho direto da origem ao destino (pois ele sempre existe), resultando em desempenho idêntico ao Dijkstra.

- **Grafos Esparsos**: Grafos com *poucas* arestas em relação ao número de vértices. Nos experimentos, as instâncias reais são grafos esparsos:

  - Instância 10k: 10.000 vértices e 123.462 arestas (densidade $\approx$ 0,12%)
  - Instância 1M: 1.000.000 vértices e 15.172.126 arestas (densidade $\approx$ 0,0015%)

  Para comparação, se fossem grafos completos dirigidos teriam aproximadamente 100 milhões e 1 trilhão de arestas, respectivamente. Neste cenário, a heurística gulosa **falha drasticamente** porque frequentemente não consegue alcançar vértices que não estão diretamente conectados ao caminho guloso inicial.

É importante esclarecer que a heurística gulosa **não é uma melhoria** em relação ao Dijkstra. A comparação visa evidenciar que economia no número de comparações não tem valor quando o algoritmo não resolve o problema corretamente (não alcança todos os vértices acessíveis).

3.6.1 Grafos Completos Dirigidos

A Tabela 5 compara diretamente o desempenho de ambos os algoritmos em grafos completos dirigidos.

Tabela 5: Comparação entre Dijkstra e Heurística Gulosa em grafos completos dirigidos.

Vértices (n)	**Dijkstra (Comparações)**	**Gulosa (Comparações)**
4	12	12
10	90	90
50	2.450	2.450
100	9.900	9.900
500	249.500	249.500
1000	999.000	999.000

Em grafos completos dirigidos, não há diferença entre os algoritmos: ambos apresentam **desempenho idêntico**, realizando exatamente $n(n-1)$ comparações para cada valor de n.

A Figura 6 ilustra graficamente a comparação entre os dois algoritmos nos grafos completos dirigidos testados.

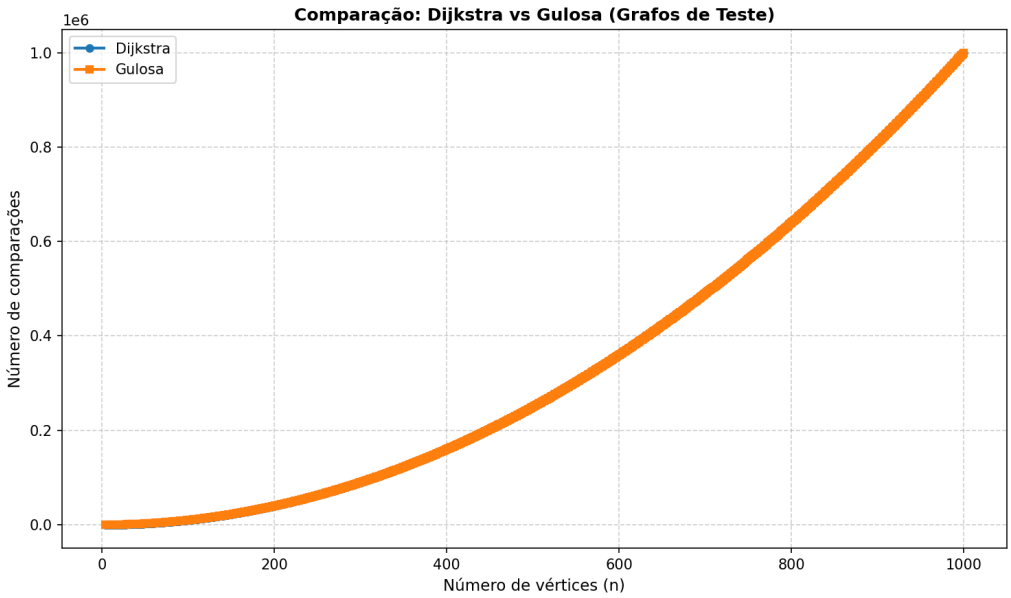

Figura 6: Comparação gráfica entre Dijkstra e Heurística Gulosa em grafos completos dirigidos. Ambos apresentam comportamento idêntico seguindo $C(n) = n(n-1)$.

3.6.2 Instâncias de Grafos Esparsos Dirigidos

A Tabela 6 compara o desempenho dos algoritmos nas instâncias de grafos esparsos direcionados.

Tabela 6: Comparação entre Dijkstra e Heurística Gulosa em instâncias de grafos esparsos dirigidos.

Instância	**Dijkstra**		**Gulosa**	
	Comparações	**Vértices**	**Comparações**	**Vértices**
10k vértices	123.462	10.000 (100%)	424	28 (0,28%)
1M vértices	15.172.126	1.000.000 (100%)	1.847	120 (0,012%)

Nas instâncias de grafos esparsos dirigidos, observa-se uma diferença drástica entre os algoritmos: a heurística gulosa realiza 99,66% (10k) e 99,99% (1M) menos comparações que Dijkstra, porém alcança apenas 0,28% e 0,012% dos vértices, respectivamente, enquanto Dijkstra alcança 100% dos vértices em ambas as instâncias.

4 Conclusão

Este trabalho comparou o algoritmo de Dijkstra com uma heurística gulosa para o problema de caminho mínimo em grafos direcionados ponderados. Os experimentos demonstraram diferenças fundamentais entre as abordagens:

Em **grafos completos dirigidos**, ambos realizaram $n(n-1)$ comparações, mas apenas Dijkstra garante caminhos mínimos ótimos. O número de comparações segue exatamente a fórmula $C(n) = n(n-1)$, com crescimento quadrático $O(n^2)$, compatível com a análise teórica usando heap binário.

Em **grafos esparsos dirigidos**, a diferença é crítica: a heurística gulosa realizou 99,66% a 99,99% menos comparações, mas alcançou apenas 0,28% (28/10.000) e 0,012% (120/1.000.000) dos vértices. Em contraste, Dijkstra alcançou 100% dos vértices em ambas as instâncias, realizando aproximadamente uma comparação por aresta (15.172.126 comparações para 15.172.126 arestas na instância 1M).

A economia de 99% nas comparações não tem valor quando o algoritmo falha em encontrar 99% da solução. A heurística gulosa demonstra inadequação total para o problema SSSP em grafos esparsos: decisões locais (escolher sempre a menor aresta) não garantem otimização global, levando a becos sem saída que interrompem prematuramente a exploração do grafo.

Conclui-se que Dijkstra é o algoritmo adequado para SSSP com pesos não-negativos, oferecendo complexidade $O((V+E)\log V)$, garantia de corretude e exploração completa do grafo. A superioridade é evidente em grafos esparsos, onde $|E| \ll |V|^2$, com número de comparações proporcional às arestas, não aos vértices ao quadrado.

O código-fonte completo está disponível no notebook Jupyter anexado, com implementações detalhadas e experimentos reproduzíveis.

Referências

[1] CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

[2] ZIVIANI, Nivio. *Projeto de algoritmos com implementações em Java e C++*. 3. ed. São Paulo: Cengage Learning, 2013.

[3] CAMPOS, Fernando de Souza. *Grafos e algoritmos computacionais*. São Paulo: LTC, 2010.

[4] HAGBERG, Aric A.; SCHULT, Daniel A.; SWART, Pieter J. Exploring network structure, dynamics, and function using NetworkX. In: *Proceedings of the 7th Python in Science Conference (SciPy)*, 2008, p. 11-15.